

Requerimientos de infraestructura para AWS

Solicitud de créditos y GPU · 2026-08-25 · Alcance: hosting de entrenamiento, simulador web y app multiusuario

Nota de idioma: los docs de este repo son en inglés por defecto. Este es una excepción explícita — va dirigido al director del GIAR y a AWS.

1. Qué es la plataforma, en tres partes

La app tiene tres cargas de trabajo con perfiles de costo muy distintos. Tratarlas como una sola "instancia grande" sobrestima el gasto — casi todo el costo real está concentrado en una sola de las tres.

PARTE	QUÉ HACE	PERFIL DE CÓMPUTO	COSTO RELATIVO
A. Entrenamiento	Corre RL (PPO) para entrenar/afinar policies de locomoción y motion-tracking	GPU intensivo, jobs cortos en ráfaga (minutos~1h)	Alto — casi todo el gasto
B. Simulador en vivo	Visualización 3D del robot corriendo una policy ya entrenada (inferencia, no entrenamiento)	Liviano, CPU alcanza, sin GPU	Bajo
C. Web app	Sitio, login, multiusuario, catálogo de policies, compartir/publicar	Web estándar: API + DB + estático	Bajo

Hoy A corre local (CPU en laptops) y en Kaggle (GPU gratuita, cola compartida, sin control de prioridad). B y C corren local vía Docker Compose. Nada está hosteado en la nube todavía — este documento es la base para migrar.

2. Parte A — Entrenamiento (la que necesita GPU)

- Cada training job es una corrida PPO de un modelo pequeño, no un LLM: minutos a ~1 hora en una GPU chica (clase T4/A10G) — no se necesita una GPU de gama alta tipo A100/H100.
- Referencia real: el free tier de Kaggle (GPU Tesla P100, Pascal) ya alcanza para entrenar — el piso de hardware requerido es bajo.
- Hoy no hay backend NVIDIA propio implementado (ni local ni cloud): son placeholders reservados en el código para esta migración.

Punto crítico de diseño: el entrenamiento debe pasar por una cola con concurrencia limitada ("semáforo"), no ejecutarse indiscriminadamente — ver §5.

3. Parte B — Simulador (viewer en la web)

- Corre inferencia (forward-only) sobre un checkpoint ya entrenado — no reentrena nada. Demostrado corriendo 100% en CPU, incluso en una Mac sin GPU.

- Cada sesión de usuario viendo el robot caminar necesita ~1-2 vCPU y poca RAM. No requiere GPU.
- Escala horizontalmente sin fricción: contenedores livianos, uno por sesión activa.

4. Parte C — Web app (login, multiusuario, futuro)

- Próximo hito: login + cuentas + que cada usuaria pueda crear/guardar/compartir sus propias policies.
- Necesita: base de datos (usuarios, catálogo, metadata de runs), storage de archivos (checkpoints de decenas de MB), autenticación, API, frontend estático.
- Es la parte más barata y más estándar de las tres.

5. El problema a resolver: 200 personas, ¿cuánta GPU en simultáneo?

Escenario de referencia: una innovaton con ~200 participantes queriendo entrenar su propia policy. Si todos entrenan a la vez sin control, el pico de demanda de GPU es indefendible en costo y en cupo de servicio de AWS.

OPCIÓN	DESCRIPCIÓN	GPUS SIMULTÁNEAS	VIABILIDAD
Sin límite	Los 200 entrenan al mismo tiempo, sin cola	~200	No viable — ni cupo AWS ni costo se justifican para jobs de minutos
Mitad simultánea	Hasta 100 en paralelo	~100	Sigue siendo desproporcionado para la duración real de un job
Un slot único	Un solo training corriendo, resto en cola estricta	1	Viable en costo, pero cola total ronda 70-130h — muy lento para un evento de un día
Pool acotado con cola	N jobs en paralelo (6-8), resto en cola automática, 1 job activo por usuario	6-8	Recomendada — balancea costo, cupo AWS y tiempo del evento

Cálculo de referencia (opción recomendada): 200 personas × 1 policy × ~30 min promedio = ~100 GPU-hora de trabajo total. Con un pool autoescalado de 6-8 GPUs en paralelo durante una ventana de evento de ~12-16 horas, el evento entero se cubre cómodo, con margen para reintentos.

Controles a implementar junto con esto (no son solo infraestructura, son producto)

- Un job activo por usuario a la vez.
- Límite de duración/iteraciones por job.
- Cola visible (posición, tiempo estimado).
- Autoscaling del pool de GPU: sube con demanda, baja a 0 sin ella — evita pagar GPU ociosa fuera de eventos.

6. Propuesta de arquitectura en AWS

COMPONENTE	SERVICIO AWS SUGERIDO	NOTAS
Workers de entrenamiento (A)	EC2 <code>g5.xlarge</code> / <code>g4dn.xlarge</code> (1 GPU c/u) en Auto Scaling Group, 0→8, o Spot Fleet + fallback on-demand	Spot para bajar costo — jobs cortos, tolerantes a interrupción con checkpointing
Cola de jobs	SQS + scheduler, o AWS Batch (cola + autoscaling GPU integrado)	AWS Batch es el camino de menor esfuerzo para el "semáforo" del §5
Simulador/viewer (B)	ECS Fargate, sin GPU, autoescalado por sesión	Costo marginal
Web app (C)	Amplify Hosting o S3+CloudFront (frontend); ECS Fargate o Lambda (API)	Estándar
Base de datos	RDS PostgreSQL (<code>db.t4g.micro</code> / <code>small</code>)	Usuarios, catálogo, metadata
Storage de policies	S3	Archivos chicos, tráfico bajo
Auth	Cognito (o Auth0 si ya se usa)	Login + multiusuario

7. Lo que le pedimos a AWS

- Cupo de servicio (service quota) GPU** familia `g5` / `g4dn` : al menos **8 GPUs concurrentes** en la región elegida (el default de una cuenta nueva suele ser 0-1).
- Créditos** para cubrir:
 - Cómputo GPU bajo demanda/spot para entrenamiento: ~100-150 GPU-hora por evento tipo innovaton (200 participantes), más uso continuo bajo del equipo fuera de eventos — presupuestar **~300-500 GPU-hora/mes** de margen.
 - Cómputo liviano (Fargate/EC2 chico) para simulador y web: ~USD 50-150/mes.
 - RDS + S3 + transferencia de datos: ~USD 20-50/mes.
- Sin monto exacto todavía — no hay facturación histórica en AWS (venimos de local + Kaggle gratuito). Pedimos que el otorgamiento sea revisable/ampliable tras 1-2 meses de datos reales de uso.

8. Resumen ejecutivo

Necesitamos: (1) cupo de GPU en AWS (familia g5/g4dn, ~8 concurrentes) para entrenamiento RL de policies — jobs cortos (minutos a ~1h), sin necesidad de GPU de gama alta; (2) cómputo liviano sin GPU para el simulador web y la app (login/multiusuario); (3) créditos que cubran ~300-500 GPU-hora/mes estimadas, más ~USD 100-200/mes de infraestructura liviana. El entrenamiento estará limitado por una cola con concurrencia acotada (no todos entrenan a la vez), así que el pico de gasto es predecible y no escala 1:1 con la cantidad de usuarios.